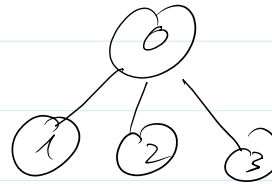
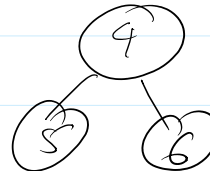
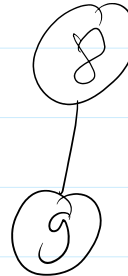
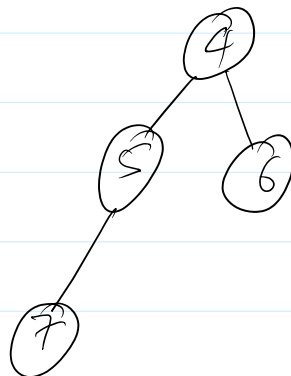
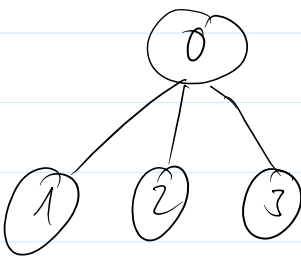


Tuần 1

Bài 4,

Tập 0, 1, 2, 3 có parent = 0 $\Rightarrow$ Tập 4, 5, 6 có parent = 4 $\Rightarrow$ parent[7] = 5 $\Rightarrow$ parent[8] = 8 $\Rightarrow$ 

Vậy có tập cây:



Vì nhánh gốc 8 có size 2 nên định x để parent[8] = x

thì tại thời điểm này size[x] $\geq$ size[8] = 2 $\Rightarrow$ Loại 1, 2, 3, 6, 9, 7 không thể là parent[8].

⊕ Xét x = 0, size[0] = 4 > size[8] = 2

 $\Rightarrow$ x = 0 có thể là parent[8]

⊕ Xét x = 4, size[4] = 4 > size[8] = 2

 $\Rightarrow$ x = 4 có thể là parent[8]

$\Rightarrow x = 4$ có thể là parent [8]

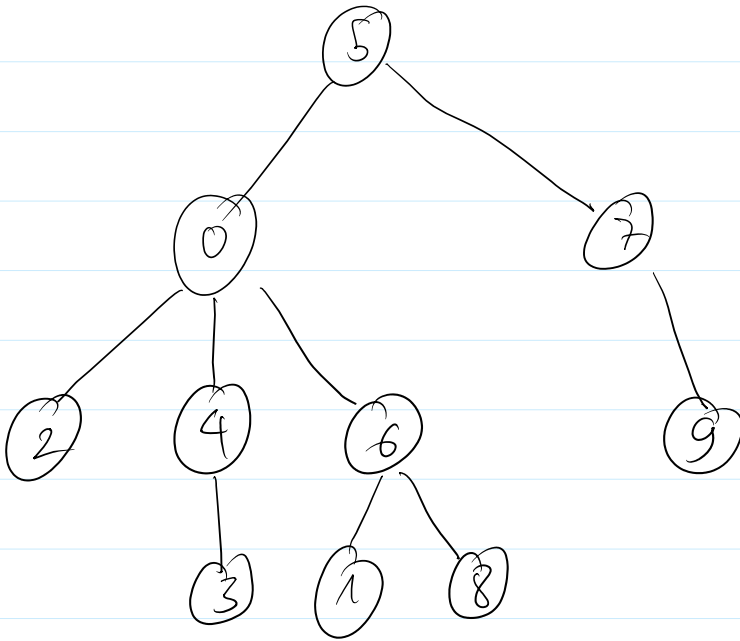
⑧ Xét $x = 5$. $size[5] = 2 > size[8] = 2$

Tuy nhiên $size[5] + size[8] = 4$

size này lớn hơn size của gốc 4 nên vô lý.

$\Rightarrow parent[8] = \{0, 4, 8\}$

Bài 5/



Nếu đỉnh v làm con đỉnh u thì $size[v] \leq size[u]$ tại thời điểm gộp. Tức size của cây sau khi gộp có kích thước ít nhất $2 \times size[v]$. Để đỉnh u nhận thêm sau đó thì $size[parent[v]] \geq 2 \times size[v]$ với mọi v không phải đỉnh.

Có thể thấy $size[0] = 7$, $size[5] = 10$

$size[5] < 2 \times size[0] \Rightarrow 5$ không thể là cha của 0

Tuần 2

Bài 1.4.16

```
import java.util.Arrays;
```

```
public class prob_1_4_16 {
```

```
/**
```

```
    Tìm và in ra cặp số có giá trị gần nhau nhất trong mảng.
```

Phân tích độ phức tạp thời gian (Trường hợp tồi nhất):

1. Sắp xếp mảng: Phương thức `Arrays.sort()` cho kiểu nguyên thủy trong Java sử dụng Giả sử sử dụng một thuật toán sắp xếp $O(N \log N)$ chuẩn, bước này mất $O(N \log N)$.
2. Duyệt qua mảng: Chúng ta lặp $N-1$ lần, thực hiện các phép toán $O(1)$ (trừ, so sánh, gán) trong mỗi vòng lặp. Bước này mất $O(N)$.
3. Tổng độ phức tạp thời gian: $O(N \log N) + O(N) = O(N \log N)$.

Độ phức tạp không gian: $O(1)$ hoặc $O(\log N)$ tùy thuộc vào thuật toán sắp xếp yêu cầu bộ nhớ phụ trợ hay không.

```
*/  
public static void findClosestPair(double[] a) {  
    if (a == null || a.length < 2) {  
        System.out.println("Mảng phải chứa ít nhất 2 phần tử.");  
        return;  
    }  
  
    Arrays.sort(a);  
  
    double minDiff = Double.MAX_VALUE;  
    double num1 = 0;  
    double num2 = 0;  
  
    for (int i = 0; i < a.length - 1; i++) {  
        double diff = Math.abs(a[i + 1] - a[i]);  
        if (diff < minDiff) {  
            minDiff = diff;  
            num1 = a[i];  
            num2 = a[i + 1];  
        }  
    }  
  
    System.out.println("Cặp gần nhất là: " + num1 + " và " + num2 + " với hiệu là " + minDiff);  
}  
  
public static void main(String[] args) {  
    double[] a = {10.5, 2.1, 8.4, 2.3, 15.0, 5.5};  
    System.out.println("Mảng ban đầu: " + Arrays.toString(a));  
    findClosestPair(a);  
}  
}
```